Ders 22 Genişletilmiş Notu: Topolojik Sıralama: DFS Pop Sırası ve Azalt ve Fethet Yaklaşımı

Ders 22 Genişletilmiş Notu: Topolojik Sıralama: DFS Pop Sırası ve Azalt ve Fethet Yaklaşımı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. DFS ile Topolojik Sıralama: Hangi Sıra Kullanılmalı?
- 2. İkinci Örnek: Altı Yazılımın Kurulum Sırası
- 3. Azalt ve Fethet ile Topolojik Sıralama
- 4. Döngülü Grafta Topolojik Sıralama Yapılamaz
- Örnek Sorular

1. DFS ile Topolojik Sıralama: Hangi Sıra Kullanılmalı?

Bu kayıt, bir önceki kayıttaki aradan sonra devam eden kısa bir oturumdur. Hoca, ders 21'in sonunda açık bıraktığı soruyla başladı: Topolojik sıralamanın (topological sorting) sonucu ne olmalı ve DFS ile grafı gezerken hangi aşamadaki bilgiyi kullanmalıyız?

Ders 21'deki örnek, ders ön koşullarını gösteren beş düğümlü yönlü bir graftı: C1 → C3, C2 → C3, C3 → C4, C3 → C5, C4 → C5. Bir $u \to v$ kenarı "u, v'den önce alınmalı" demektir. DFS, C1'den başlayıp eşitlikte küçük numaralı düğümü seçerek gezildiğinde yığına (stack) önce C1, C3, C4, C5 eklenir. C5'in gidilecek komşusu kalmayınca yığından sırasıyla C5, C4, C3, C1 çıkar. Yığın boşalınca ziyaret edilmemiş C2 eklenir ve o da çıkar.

1.1 Push sırası neden olmaz?

Derste bir öğrenci, düğümleri yığına eklenme (push) sırasıyla yazdırmayı önerdi: Yığın C5'e kadar dolunca C1, C3, C4, C5 yazdırılır, yığın boşalınca C2 eklenip yazdırılır. Hoca bu sırayı açıkça yazdı: C1, C3, C4, C5, C2. Sonra sordu: Bu durumda yapı bozulmuyor mu?

Bozulur. C2, C3'ün ön koşuludur, yani C3'ten önce gelmelidir. Push sırasında ise C2 en sona düşer. Hoca da C1 ile C2'nin başta olması gerektiğini söyledi ve bu tartışmanın tam istediği şey olduğunu belirtti.

1.2 Pop sırasının tersi

Başka bir öğrenci sıranın tersini önerdi: En son C2 çıktığına göre önce C2 yazdırılır, sonra C1, C3, C4, C5. Hoca bunu "push değil, popping-off order'ın (çıkarma sırasının) tersi" diye adlandırdı ve tahtada kırmızıyla izledi:

- Çıkarma (pop) sırası: C5, C4, C3, C1, C2.
- Bu sıra ters yönde okunursa: C2, C1, C3, C4, C5.

Bu sıra bütün ön koşulları sağlar ve geçerli bir topolojik sıralamadır. Hocanın sonucu: Pop sırasının tersiyle topolojik sıralama yapma şansımız var, push sırasıyla yok.

> **Ek bilgi (derste anlatılmadı):** Pop sırasının tersinin neden doğru olduğu şöyle açıklanır. DFS bir düğümü, ondan ulaşılabilen bütün düğümler bittikten sonra yığından çıkarır. Bu yüzden döngüsüz bir grafta $u \to v$ kenarı varsa v, u'dan önce çıkar. Sıra ters çevrilince u, v'den önce gelir; bu da tam olarak istenen koşuldur.

2. İkinci Örnek: Altı Yazılımın Kurulum Sırası

Konunun daha iyi oturması için hoca, derse bağlanmadan önce hazırladığı altı düğümlü ikinci bir grafı gösterdi. Bu kez düğümler, birbirine bağımlı altı yazılımdı: Hangi yazılımın hangisinden önce yüklenebileceği graf olarak ifade edilmişti.

Kısa notta kareden okunan kenarlar şunlardır: C1 → C2, C1 → C3, C2 → C4, C3 → C4, C3 → C5, C4 → C5, C4 → C6.

2.1 DFS adımları

DFS, C1'den başlar ve eşitlikte küçük numaralı komşuyu seçer:

1. Yığına C1, sonra C2, sonra C4, sonra C5 eklenir.
2. C5'in gidilecek komşusu yoktur. C5 çıkarılır (pop).
3. Yığının tepesinde C4 vardır. C4'ün ziyaret edilmemiş komşusu C6 yığına eklenir.
4. C6'nın da komşusu yoktur. C6 çıkarılır.
5. C4'ün başka komşusu kalmadı. C4 çıkarılır, ardından C2 çıkarılır.
6. Yığında C1 kalır. Hoca "C1 çıkacak mı?" diye sordu. Bir öğrenci doğru cevabı verdi: C1 çıkmaz, çünkü C1'in ziyaret edilmemiş komşusu C3 vardır. C3 yığına eklenir.
7. Artık bütün düğümlere erişilmiştir. C3 çıkarılır, en son C1 çıkarılır.

Pop sırası: C5, C6, C4, C2, C3, C1.

Bu sıranın tersi kurulum sırasını verir: önce C1, sonra C3, C2, C4, C6 ve C5 yüklenir. Python ile kontrol edildi: Bu sıra bütün kenar koşullarını sağlar. Push sırası ise C1, C2, C4, C5, C6, C3 olur ve geçersizdir (C3, C4'ten önce gelmelidir).

Hoca, sıranın değişebileceğini vurguladı. Topolojik sıralama tek değildir; bulunan sıra bir çözümdür. Python ile bütün sıralar denendiğinde bu grafın 4 geçerli topolojik sırası olduğu görüldü.

3. Azalt ve Fethet ile Topolojik Sıralama

Hoca bir soru sordu: Bu problemi başka hangi algoritmik yaklaşımla çözebiliriz? İpucu olarak bunun 3-4 hafta önce konuşulan bir bakış açısı olduğunu söyledi. Bir öğrenci genişlik öncelikli aramayı (breadth-first search) önerdi. Hoca, ona da vakit kalırsa bakılabileceğini, ama problemin doğasına daha uygun bir çözüm olduğunu söyledi: böl ve fethete (divide and conquer) yakın olan azalt ve fethet (decrease and conquer) yaklaşımı.

3.1 Azalt ve fethet mantığı

Hoca azalt ve fethetin mantığını sordu. Bir öğrenci "problemi daha küçük parçalara bölüp parçaları teker teker çözmek ve birleştirmek" diye cevap verdi; hoca teşekkür edip devam etti.

> **Ders sonrası not:** Öğrencinin verdiği ve hocanın kabul ettiği tanım, böl ve fethetin (divide and conquer) tanımıdır. Azalt ve fethet, problemi bir adımda tek bir küçük örneğe indirir ve yalnız onu çözer. Küçültme miktarı sabit bir sayı (bu derste olduğu gibi bir düğüm), sabit bir oran (ikili aramada yarıya) ya da değişken olabilir.

Başka bir öğrenci, düğümleri C1, C2, C3 ve C4, C5, C6 diye iki gruba ayırmayı önerdi. Hoca, bunun böl ve fethet gibi ikiye bölmek olduğunu söyledi. İkili aramada (binary search) azaltmayı ikiye bölerek yapıyorduk. Burada ise ikiye bölmeden, adım adım ilerlemek istiyoruz.

3.2 Başlangıç düğümü nasıl bulunur?

Hoca soruyu daralttı: İlk adımda kesinlikle alınabilecek ders, ya da yüklenebilecek yazılım hangisidir? Öğrenciler C1 dedi.

Soru: C1'in ilk adım olduğunu grafa bakınca nereden anlarız? Bir öğrenci "baş (head) düğüm olduğu için" dedi. **Cevap:** Hoca buna karşı çıktı: Grafı ters yüz edip başka türlü çizersem C1 artık "baş" görünmez. Doğru ölçütü başka bir öğrenci verdi: C1'e giren hiçbir yol (kenar) yoktur. Kendisine gelen herhangi bir kenar olmadığı için C1 otomatik olarak başlangıç noktası olabilir.

3.3 Problemi küçültmek

C1 alındıktan sonra onunla işimiz biter. Bir öğrencinin dediği gibi, artık C1'in olmadığı grafın topolojik sıralamasına bakarız. C1 kenarlarıyla birlikte graftan çıkar; problem küçülmüştür. Adımlar şöyle ilerler:

1. C1 alınır ve çıkarılır. Kalan grafta kendisine giren kenarı olmayan düğümler C2 ve C3'tür. İkisi de alınabilir. Hoca küçük numaralı olanı, C2'yi seçti.
2. C2 çıkarılınca alınabilecek tek düğüm C3'tür. C4 alınamaz, çünkü C4'ün hâlâ C3'ten gelen bağlantısı vardır.
3. C3 alınır. Kalan grafta doğal olarak C4 alınabilir.
4. C4 alındıktan sonra birbirinden bağımsız iki düğüm kalır: C5 ve C6. İkisi de alınabilir; hoca sırayı bozmamak için önce C5'i, sonra C6'yı aldı.

Sonuç: C1, C2, C3, C4, C5, C6. Bu sıra DFS'in verdiği C1, C3, C2, C4, C6, C5 sırasından farklıdır, ama bu da geçerli bir çözümdür (python ile doğrulandı).

> **Hocanın vurgusu:** Topolojik sıralama, azalt ve fethet yaklaşımına çok güzel bir örnektir. Grafta kendisine giriş olmayan düğümü başlangıç noktası olarak belirlersiniz. Onu çıkardıktan sonra kalan grafta yine kendisine giriş olmayan bir düğüm varsa yolunuza devam edersiniz.
>
> **Ek bilgi (derste anlatılmadı):** Bu kaynak silme yöntemine Kahn algoritması da denir. Her düğümün giriş derecesi (in-degree) bir dizide tutulur ve giriş derecesi sıfır olan düğümler bir kuyruğa konursa, her düğüm ve her kenar bir kez işlenir. Böylece yöntem $\Theta(|V| + |E|)$ sürede çalışır.

4. Döngülü Grafta Topolojik Sıralama Yapılamaz

Hoca son olarak şunu sordu: DFS'i değil de azalt ve fethet yaklaşımını kullanıyorsak, bir grafın çözülemez olduğunu, yani topolojik sıralamasının olmadığını nasıl anlarız?

Bunun için dört düğümlü bir graf çizdi. Kısa notta kareden okunan haliyle kenarlar C1 → C3, C3 → C4, C4 → C2'dir. Hoca önce bu grafta döngü (cycle) olup olmadığını sordu; bir öğrenci olmadığını söyledi ve hoca onayladı. Sonra döngü oluşturmak için bir bağlantı daha ekledi: C2 → C1. Artık C1 → C3 → C4 → C2 → C1 bir döngüdür.

Bir öğrenci cevabı verdi: Başlayabileceğimiz bir düğüm yoktur. Kendisine yönlendirilmiş kenarı olmayan, "boşta" bir düğüm bulunamaz. Döngü eklenmeden önce C1 bir kaynak düğümdü; C2 → C1 kenarı eklenince her düğüme bir kenar girer. Python ile denendiğinde de döngüsüz halde sıra C1, C3, C4, C2 bulunurken döngülü halde ilk adımda kaynak düğüm kalmadığı görüldü.

Hoca, bunu anlamak için DFS algoritmasına gerek olmadığını vurguladı. Yalnızca her düğüme (C1, C2, C3, … hepsine) bakıp bir komşudan kendisine gelen bir ok olup olmadığını kontrol etmek yeterlidir. Böyle bir düğüm bulunamadığı anda sorunun cevabı bellidir: Verilen yazılımlarla ya da verilen ders programıyla izlenebilecek bir yol yoktur.

> **Ek bilgi (derste anlatılmadı):** DFS ile de döngü tespit edilebilir. Gezinti sırasında yığında hâlâ duran bir düğüme giden bir kenar (geri kenar, back edge) bulunursa yönlü grafta döngü vardır. Bu durumda pop sırasının tersi geçerli bir topolojik sıralama vermez.

Örnek Sorular

Soru 1 (derste sözlü, 00:01:12): Beş düğümlü ders ön koşulu grafında (C1 → C3, C2 → C3, C3 → C4, C3 → C5, C4 → C5), DFS sırasında düğümleri yığına eklenme (push) sırasıyla yazdırmak geçerli bir topolojik sıralama verir mi? Vermiyorsa hangi sıra kullanılmalıdır?

> **Çözüm:** Push sırası C1, C3, C4, C5, C2'dir ve geçersizdir, çünkü C2, C3'ten önce gelmelidir. Pop sırası C5, C4, C3, C1, C2'dir; bunun tersi C2, C1, C3, C4, C5 geçerli bir topolojik sıralamadır.

Soru 2 (derste sözlü, 00:05:01): Altı yazılımlı örnekte DFS yığınında C1 kaldığında C1 hemen çıkarılır mı?

> **Çözüm:** Hayır. C1'in ziyaret edilmemiş komşusu C3 vardır. Önce C3 yığına eklenir ve çıkarılır, sonra C1 çıkarılır. Pop sırası C5, C6, C4, C2, C3, C1; kurulum sırası C1, C3, C2, C4, C6, C5 olur.

Soru 3 (derste sözlü, 00:06:26): Topolojik sıralama problemi DFS dışında hangi algoritmik yaklaşımla çözülebilir?

> **Çözüm:** Azalt ve fethet (decrease and conquer) ile. Kendisine giren kenarı olmayan bir düğüm seçilir, sıraya yazılır ve graftan çıkarılır; kalan grafta aynı işlem tekrarlanır. Altı yazılımlı örnekte sonuç C1, C2, C3, C4, C5, C6 olur.

Soru 4 (derste sözlü, 00:10:02): Grafa bakınca ilk alınabilecek dersin C1 olduğunu nereden anlarız?

> **Çözüm:** C1'e giren hiçbir kenar yoktur. "Baş düğüm olması" ölçüt değildir, çünkü graf farklı çizilince bu görünüş değişir.

Soru 5 (derste sözlü, 00:14:16): Azalt ve fethet ile ilerlerken bir grafın topolojik sıralamasının olmadığını nasıl anlarız?

> **Çözüm:** Kendisine gelen kenarı olmayan bir düğüm bulunamazsa grafta döngü vardır ve topolojik sıralama yapılamaz. Bunun için DFS gerekmez; her düğüme gelen bir kenar olup olmadığına bakmak yeterlidir.

Dersin Toparlaması

Bu kısa oturumda DFS tabanlı topolojik sıralama tamamlandı. Yığına ekleme (push) sırası ön koşulları bozabilir; doğru sonuç, düğümlerin yığından çıkarılma (pop) sırasının tersidir. Bu yöntem altı yazılımın kurulum sırası örneğinde adım adım uygulandı ve topolojik sıralamanın tek olmadığı vurgulandı.

Ardından aynı problem azalt ve fethet yaklaşımıyla çözüldü: Kendisine giren kenarı olmayan düğüm seçilir, graftan çıkarılır ve küçülen grafta işlem tekrarlanır. Bu yaklaşım, döngüyü de doğal olarak yakalar: Hiç kaynak düğüm kalmamışsa grafta döngü vardır ve izlenebilecek bir ders ya da kurulum sırası yoktur. Hoca, genişlik öncelikli aramaya vakit kalırsa daha sonra bakılabileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.